

Analisis tecnico del bug de desalineacion semantica en PCZT

Documento interno de auditoria tecnica

12 de mayo de 2026

Índice

1. Resumen ejecutivo	2
2. Contexto	2
2.1. ¿Qué es PCZT?	2
2.2. Hipótesis inicial	3
3. Preguntas que guiaron la investigación	3
4. Mapa de fuentes de verdad	4
4.1. Fuente de verdad criptografica	4
4.2. Fuente de verdad de presentación	4
5. Flujo vulnerable	4
5.1. Creacion del PCZT	4
5.2. Firma	5
6. Por qué la PoC sigue siendo válida sin create_pczt_from_proposal	5
7. Por qué hace falta conservar PcztRecipient::External	6
8. Es solo misuse del rol Updater?	6
8.1. Extracción y almacenamiento	7
9. Hipótesis exacta del exploit	7
10.PoC ejecutable	7
10.1. Test agregado	7
10.2. ¿Qué hace el test?	8
10.3. Comando de ejecución	8
10.4. Salida observada	8
10.5. Resultado semántico exacto	9
10.6. Interpretación	9
11.¿Por qué las firmas siguen siendo válidas?	9
12.Preguntas que nos hicimos hasta llegar al PoC	9

13.Puntas a seguir investigando	10
13.1. Signer deception real frente a una UI humana	10
13.2. Orchard	10
13.3. zcash_client_memory	10
13.4. Zallet	11
13.5. zcashd	11
14.Clasificación técnica	12
15.Recomendaciones	12

1. Resumen ejecutivo

Este documento describe, de punta a punta, el proceso que seguimos para encontrar, validar y convertir en PoC un bug semantico en el subsistema PCZT (*Partially Constructed Zcash Transaction*) dentro de `librustzcash`.

La conclusion central es la siguiente:

Es posible construir un PCZT valido cuyo output shielded comprometido paga al recipient B, pero cuya metadata auxiliar hace que la capa de reconstruccion de historial/salidas enviadas registre y exponga al recipient A.

En otras palabras:

- **Recipient comprometido criptograficamente: B**
- **Recipient mostrado/almacenado en sent history: A**
- **La transaccion sigue siendo valida y se acepta: sí**

Lo que **si** demostramos:

- un flujo end-to-end dentro de `zcash_client_backend` donde:
 - se crea un PCZT válido,
 - se muta solo metadata de presentación,
 - se firman los spends,
 - se extrae la transacción final,
 - y la capa de almacenamiento de salidas enviadas persiste un recipient diferente del comprometido.

Lo que no demostramos todavía (y vamos a intentar hacer) es:

- que una UI real de confirmacion de firma muestre A al usuario humano antes de firmar;
- que un wallet de produccion externo a este repo renderice esta metadata como fuente autoritativa en su pantalla de aprobación.

2. Contexto

2.1. ¿Qué es PCZT?

PCZT es el formato de transaccion parcialmente construida de Zcash. La idea es separar la construccion de una transaccion en roles:

- `Creator`
- `Updater`
- `IoFinalizer`
- `Prover`
- `Signer`
- `Combiner`

- Redactor
- SpendFinalizer
- TransactionExtractor
- Verifier

Cada rol agrega, valida, poda o transforma información.

2.2. Hipótesis inicial

La hipótesis que queríamos probar era:

Un coordinador malicioso puede hacer que un participante vea semánticamente “recipient A” mientras la salida realmente comprometida por la transacción paga a “recipient B”.

La forma mas concreta de esto en el código era:

- recipient comprometido en campos shielded del output;
- recipient de presentación en campos auxiliares como:
 - `user_address`
 - metadata propietaria `PROPRIETARY_OUTPUT_INFO`
 - `PcztRecipient::External`

3. Preguntas que guiaron la investigación

Estas fueron las preguntas técnicas que nos fuimos haciendo durante la búsqueda:

1. ¿Cuáles campos estan comprometidos criptográficamente y cuáles son solo metadata?
2. ¿El recipient que ve un signer o un consumidor downstream sale del output real o de metadata?
3. ¿Hay algun chequeo generico que garantice:

`user_address = recipient comprometido`

antes de firmar?

4. ¿El **Signer** de alto nivel valida la semantica de outputs o solo la validez mínima para firmar?
5. `extract_and_store_transaction_from_pczt` reconstruye el recipient desde la nota o desde metadata?
6. Si mutamos solo `user_address`, ¿las firmas siguen siendo válidas?
7. Si las firmas siguen siendo válidas, ¿el historial guardado usa A o usa B?

4. Mapa de fuentes de verdad

4.1. Fuente de verdad criptografica

Para outputs shielded, el recipient comprometido depende de:

- en Sapling:
 - `recipient`
 - `value`
 - `rseed`
 - `ciphertext`
 - `commitment`
- en Orchard:
 - `recipient`
 - `value`
 - `rho`
 - `rseed`
 - `ciphertext`
 - `commitment`

Esos campos alimentan:

- el note comprometido,
- la recuperación del output,
- la semántica on-chain real.

4.2. Fuente de verdad de presentación

Para recipients externos, en el flujo PCZT aparecen dos piezas auxiliares:

- `user_address`
- `PROPRIETARY_OUTPUT_INFO` con `PcztRecipient::External`

Estas piezas **no** estan comprometidas en sighash ni en commitments.

5. Flujo vulnerable

5.1. Creacion del PCZT

En `zcash_client_backend/src/data_api/wallet.rs`, el flujo `create_pczt_from_proposal` hace dos cosas importantes:

1. toma el recipient semántico de la propuesta;
2. lo traduce a metadata auxiliar:

- `user_address`
- `PROPRIETARY_OUTPUT_INFO`

Para outputs externos, guarda:

`PcztRecipient::External`

y tambien serializa la direccion de usuario.

5.2. Firma

El `Signer` de alto nivel:

- calcula el `shielded_sighash` desde `TransactionData<EffectsOnly>`;
- firma los spends;
- no impone un chequeo output-side de consistencia entre:
 - `recipient` comprometido,
 - y `user_address`.

Conclusión importante: si mutamos `user_address`, las firmas no cambian.

6. Por qué la PoC sigue siendo válida sin `create_pczt_from_proposal`

Una objeción posible es:

la PoC no parte siempre del helper de alto nivel `create_pczt_from_proposal`, por lo tanto tal vez solo demuestra misuse de APIs internas

No creemos que esa objeción invalide el hallazgo.

Razones:

- la PoC usa APIs públicas e intencionales del flujo PCZT:
 - `Builder::build_for_pczt`
 - `Creator::build_from_parts`
 - `IoFinalizer`
 - `Updater`
 - `Prover`
 - `Signer`
 - `SpendFinalizer`
 - `TransactionExtractor`
 - `extract_and_store_transaction_from_pczt`
- no depende de `unsafe`, ni de patching binario, ni de mutar campos privados;
- PCZT existe precisamente para escenarios multi-actor, coordinadores, hardware wallets y separación de roles;
- en ese modelo, un coordinador malicioso es un adversario natural.

Entonces, la pregunta de seguridad correcta no es:

`create_pczt_from_proposal` produce por sí sola este estado?

sino:

un caller que usa la API pública de PCZT puede construir o mutar una PCZT válida cuya semántica mostrada diverja de la semántica comprometida?

Y la respuesta actual es: sí.

7. Por qué hace falta conservar `PcztRecipient::External`

Otro punto importante: mutar solo `user_address` no alcanza por sí solo para que aparezca el recipient falso en sent history.

La razón es que `extract_and_store_transaction_from_pczt` reconstruye dos cosas por separado:

1. la `note` comprometida desde los campos shielded reales;
2. la clasificación semántica del recipient desde `PROPRIETARY_OUTPUT_INFO ->PcztRecipient`.

Para obtener un recipient externo mostrado en historial, la combinación clave es:

- `PcztRecipient::External`
- junto con `Some(user_address)`

Eso es lo que termina construyendo:

```
Recipient::External { recipient_address: addr, ... }
```

Si no preservamos `PcztRecipient::External`:

- el output puede interpretarse como interno;
- o ignorarse como dummy / irrecoverable;
- y la dirección externa falsa no emerge en el mismo path.

Por eso la PoC necesita:

- conservar la clasificación de recipient externo;
- y cambiar la dirección advisory asociada a esa clasificación.

Esto no es una precondition artificial. En el flujo honesto, `create_pczt_from_proposal` ya escribe `PcztRecipient::External` para outputs externos. La PoC solamente preserva o recrea ese mismo estado público mientras falsifica la dirección mostrada.

8. Es solo misuse del rol Updater?

La mejor lectura no parece ser:

la PoC abusa `Updater` en una forma no permitida, así que no hay bug

La lectura más precisa es:

- semánticamente, el coordinador actúa de forma deshonesto;
- técnicamente, la librería no impone un contrato que prohíba esa mutación.

Razones:

- `Updater` está expuesto como rol amplio y público;
- sus APIs permiten mutar metadata y campos del bundle;
- `IoFinalizer` congela efectos de transacción, no metadata advisory como `user_address`;

- ningún rol genérico posterior vuelve a verificar que `user_address`, `PcztRecipient::External` y el recipient comprometido sigan alineados semánticamente.

Por eso la PoC no depende de comportamiento indefinido; depende de una invariante ausente:

la librería nunca vuelve a ligar la metadata de presentación del recipient con el recipient shielded realmente comprometido

8.1. Extracción y almacenamiento

En `extract_and_store_transaction_from_pczt` ocurre el punto clave. La función:

- reconstruye el note desde los campos comprometidos;
- pero reconstruye el recipient externo para historial usando metadata:
 - `PcztRecipient::External`
 - `user_address`

En el helper `to_sent_transaction_output(...)`:

```
(PcztRecipient::External, Some(addr)) =>
  Recipient::External { recipient_address: addr, ... }
```

Conclusión importante: el recipient externo almacenado sale de metadata, no del recipient comprometido del note.

9. Hipótesis exacta del exploit

La condicion necesaria y suficiente es:

1. construir un PCZT válido que pague a B;
2. mutar solo `user_address = A`;
3. dejar:
 - recipient comprometido = B
 - ciphertext / note / commitment = intactos
4. firmar normalmente;
5. extraer/almacenar;
6. observar si el recipient almacenado es A.

10. PoC ejecutable

10.1. Test agregado

Se agregó el siguiente test:

```
data_api::wallet::tests::
extract_and_store_transaction_from_pczt_can_store_metadata_recipient_distinct_from_committed_sapling
```

Archivo:

```
zcash_client_backend/src/data_api/wallet.rs
```


10.2. ¿Qué hace el test?

1. crea una transacción válida `transparent` ->Sapling;

2. recipient comprometido real:

```
committed_recipient_addr = B
```

3. recipient falso de presentación:

```
fake_displayed_addr = A
```

4. construye el PCZT;

5. muta solo:

```
output.user_address = A
```

6. preserva:

```
PcztRecipient::External
```

7. crea proofs;

8. firma;

9. finaliza spends transparentes;

10. extrae la transaccion final;

11. recupera el recipient comprometido via OVK del sender;

12. persiste el resultado en un `MockWalletDb` instrumentado para capturar `SentTransactionOutput`;

13. compara el recipient recuperado con el recipient almacenado.

10.3. Comando de ejecución

```
cargo test -p zcash_client_backend --features pczt \
  extract_and_store_transaction_from_pczt_can_store_metadata_recipient_distinct_from_committed_sapling
  \
  -- --nocapture
```

10.4. Salida observada

```
running 1 test
test data_api::wallet::tests::
extract_and_store_transaction_from_pczt_can_store_metadata_recipient_distinct_from_committed_sapling
... ok
```

10.5. Resultado semántico exacto

El test prueba estas igualdades/desigualdades:

- recipient comprometido recuperado del tx:

```
recovered_recipient_addr == committed_recipient_addr == B
```

- recipient mostrado/almacenado:

```
displayed_recipient == fake_displayed_addr == A
```

- mismatch:

```
displayed_recipient != recovered_recipient_addr
```

10.6. Interpretación

Esto equivale a:

Committed recipient: B

Displayed/stored recipient: A

Transaction accepted: yes

11. ¿Por qué las firmas siguen siendo válidas?

Porque el campo mutado, `user_address`, es metadata auxiliar. Entonces:

- el recipient real B queda criptográficamente intacto;
- la metadata A sigue viva para presentación/almacenamiento;
- la verificación criptográfica no detecta el desalineamiento.

Tenemos un problema de fuentes de verdad:

1. la metadata de recipient externo es mutable;
2. esa metadata no esta comprometida en la firma;
3. la capa de reconstrucción/almacenado la usa como autoritativa.

12. Preguntas que nos hicimos hasta llegar al PoC

Estas fueron las preguntas mas útiles que guiaron el hallazgo:

1. ¿El recipient que ve el consumidor downstream sale del output real o de metadata?
2. ¿`user_address` está comprometido en sighash? No.
3. ¿Existe algun chequeo genérico:

```
user_address = recipient comprometido
```

antes de firmar? No.

4. ¿`extract_and_store_transaction_from_pczt` usa metadata para clasificar recipients externos? Sí.
5. Entonces, si mutamos solo `user_address`, ¿se rompe la firma? No.
6. Si no se rompe la firma, ¿se persiste A o B? Se persiste A.

13. Puntas a seguir investigando

13.1. Signer deception real frente a una UI humana

No demostramos todavía que exista, una UI de confirmación de firma que muestre `user_address` = A al usuario humano y le haga aprobar mientras el output real paga a B. Seguiremos investigando a partir de las wallets principales usadas en el ecosistema ZCash.

13.2. Orchard

Ya repetimos el mismo PoC completo para Orchard a nivel backend en `zcash_client_backend`. El test agregado es:

```
data_api::wallet::tests::  
extract_and_store_transaction_from_pczt_can_store_metadata_recipient_distinct_from_committed_orchard.
```

El resultado observado fue:

```
running 1 test  
test data_api::wallet::tests::  
extract_and_store_transaction_from_pczt_can_store_metadata_recipient_distinct_from_committed_orchard.  
... ok
```

El resultado semántico es el mismo que en Sapling:

- recipient comprometido recuperado del tx: B
- recipient mostrado/almacenado: A
- la transacción sigue siendo válida y se acepta

La diferencia práctica es que Orchard agrega más piezas comprometidas (`rho`, `rseed`), pero el punto vulnerable sigue siendo el mismo: `user_address` y la metadata propietaria quedan fuera de la parte comprometida y luego se usan como fuente autoritativa para sent history.

13.3. `zcash_client_memory`

También repetimos el hallazgo sobre una implementación concreta de storage: `zcash_client_memory`. Los tests agregados fueron:

```
testing::pool::sapling::pczt_sent_history_can_be_misled_by_user_address  
testing::pool::orchard::pczt_sent_history_can_be_misled_by_user_address
```

Resultado observado:

```
running 2 tests  
test testing::pool::sapling::pczt_sent_history_can_be_misled_by_user_address ... ok  
test testing::pool::orchard::pczt_sent_history_can_be_misled_by_user_address ... ok
```

Esto importa porque muestra que el problema no queda restringido a un `MockWalletDb` de pruebas unitarias, sino que sobrevive al menos a una implementación real de wallet store dentro de este repo.

13.4. Zallet

También construimos un harness local sobre un fork de `zcash/wallet` (*Zallet*) para medir si una capa de presentación downstream aceptaría el recipient cacheado falso.

El resultado fue:

- si sembramos en la base de datos de Zallet una transacción Orchard derivada de un PCZT malicioso,
- y la fila cacheada conserva `to_address = A`,
- la lógica de `list_transactions` devuelve A aunque el recipient recuperable del output siga siendo B.

Pero la distinción importante es:

- esto demuestra impacto en la capa de history/RPC de Zallet;
- no demuestra reachability en el send path productivo actual de Zallet;
- hoy Zallet no usa PCZT en su flujo normal de construcción/envío.

Por lo tanto, el resultado de Zallet debe interpretarse como:

extensión local del impacto a una capa downstream real de wallet history, pero no como exploit de producción confirmado contra el flujo actual de Zallet

13.5. zcashd

También revisamos si la wallet legada de `zcashd` parece heredar el mismo bug.

La conclusión actual es:

zcashd no parece heredar este bug en la misma forma

Razones:

- `zcashd` no usa hoy PCZT en su flujo normal de wallet send;
- su capa de presentación sí guarda mappings de recipient para `z_viewtransaction`;
- pero ese cache no es un string libre tipo `user_address`;
- en cambio usa un `RecipientMapping` entre el receiver concreto y una UA opcional;
- y `WriteRecipientMapping` valida que la UA cacheada contenga efectivamente ese receiver antes de persistirla.

En otras palabras:

- `zcashd` puede preferir una forma de dirección más amigable para mostrar;
- pero no parece permitir que esa forma de presentación apunte a un recipient distinto del realmente comprometido.

Por eso, a diferencia de `librustzcash` PCZT y del harness local sobre Zallet, no encontramos una extensión directa del mismo bug hacia la wallet de `zcashd`.

14. Clasificación técnica

semantic binding failure between committed shielded recipient and stored/displayed external recipient metadata in PCZT-derived sent-history flows

15. Recomendaciones

1. No usar `user_address` como fuente autoritativa de recipient externo si no fue verificada contra el recipient comprometido.
2. Recomputar recipient de salida desde campos comprometidos siempre que sea posible.
3. En el signer de alto nivel, agregar validación explícita de coherencia output-side.
4. Si se conserva metadata de presentación, marcarla explícitamente como advisory y no mezclarla con objetos de “recipient real”.
5. Reproducir el mismo hallazgo a través de `zcash_client_sqlite` cuando ese crate vuelva a compilar sin drift de API.